

System Architecture Document

SentinelCut — AI-Powered Audio/Video Profanity Censorship Platform

Document Version: 1.0 **Status:** Final for Implementation **Based On:** Product Vision v1.0, PRD v1.0, SRS v1.0, Tech Stack Justification v1.0

Diagram Notation: All diagrams below are written in Mermaid syntax. They render natively on GitHub, GitLab, Notion, and most modern Markdown viewers. If viewing in a plain-text editor, paste any block into mermaid.live to visualize it.

1. Architectural Style & Principles

SentinelCut follows a **layered, service-oriented monolith with an asynchronous background-processing subsystem** — not a full microservices split. This is a deliberate choice, justified below.

1.1 Why Not Microservices (for MVP)

Consideration	Decision Rationale
Team size	Small academic team; microservices add operational overhead (service discovery, inter-service auth, distributed tracing) disproportionate to team capacity
Language unity	Backend + AI pipeline are both Python (per Tech Stack doc); no cross-language boundary forces a service split
Deployment constraint	Free-tier hosting (Render/Railway) favors fewer, simpler deployable units
Future-proofing	The internal architecture is still modular (service layer + repository pattern + queue-based workers), so it can be decomposed into real microservices later (e.g., splitting the AI worker into its own scalable service) without a rewrite

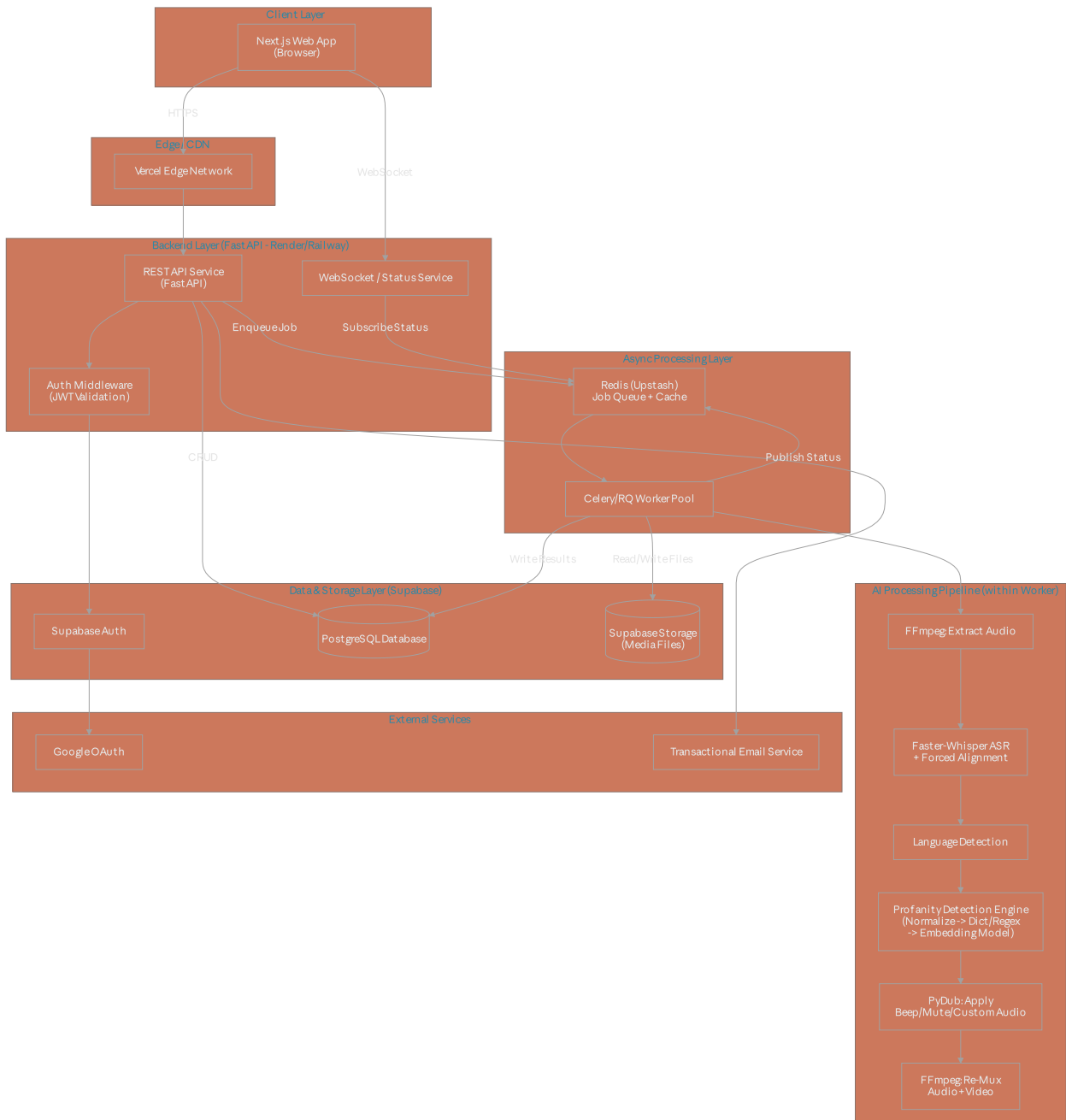
1.2 Core Architectural Principles

- Separation of Concerns:** API layer, business logic (services), data access (repositories), and AI processing are distinct, independently testable layers.
- Asynchronous Processing:** Any operation exceeding a few seconds (ASR, detection, rendering) runs as a background job via a queue — the API never blocks on AI processing.
- Interface-Driven AI Components:** The ASR engine and profanity detection engine are accessed through internal service interfaces

((SpeechRecognitionService), (ProfanityDetectionService)), allowing model swaps without touching calling code (per SRS-NFR-MAINT-01).

- **Single Source of Truth for Auth/Data/Storage:** Supabase (Postgres + Auth + Storage) unifies these three concerns, as justified in the Tech Stack document.
- **Idempotent, Resumable Jobs:** Processing jobs are designed so a retry (SRS-NFR-REL-02) does not duplicate side effects.

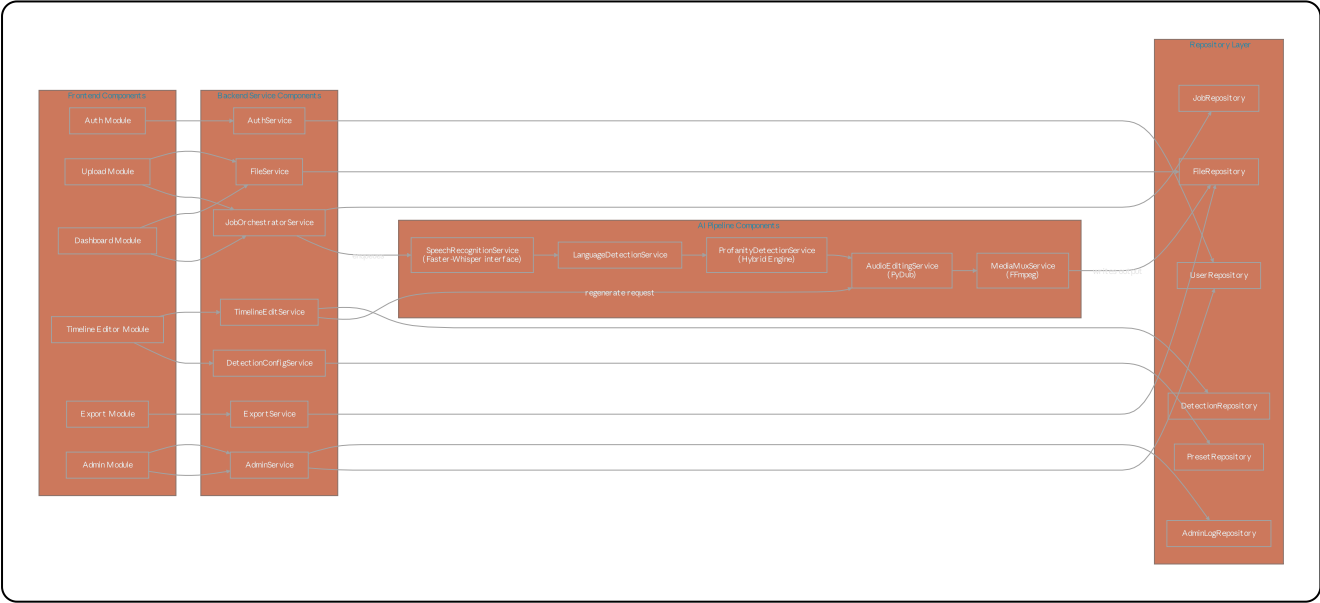
2. High-Level System Architecture



2.1 Layer Summary

Layer	Responsibility
Client Layer	Renders UI, manages local editor state, communicates via REST + WebSocket
Edge/CDN	Serves static/SSR frontend assets globally with low latency
Backend API Layer	Auth validation, request handling, job enqueueing, CRUD operations
Async Processing Layer	Decouples long-running AI work from the request/response cycle
AI Processing Pipeline	The core censorship logic — ASR, language ID, profanity detection, audio editing, muxing
Data & Storage Layer	Persists users, files, metadata, and binary media
External Services	Third-party integrations (OAuth, email)

3. Component Diagram

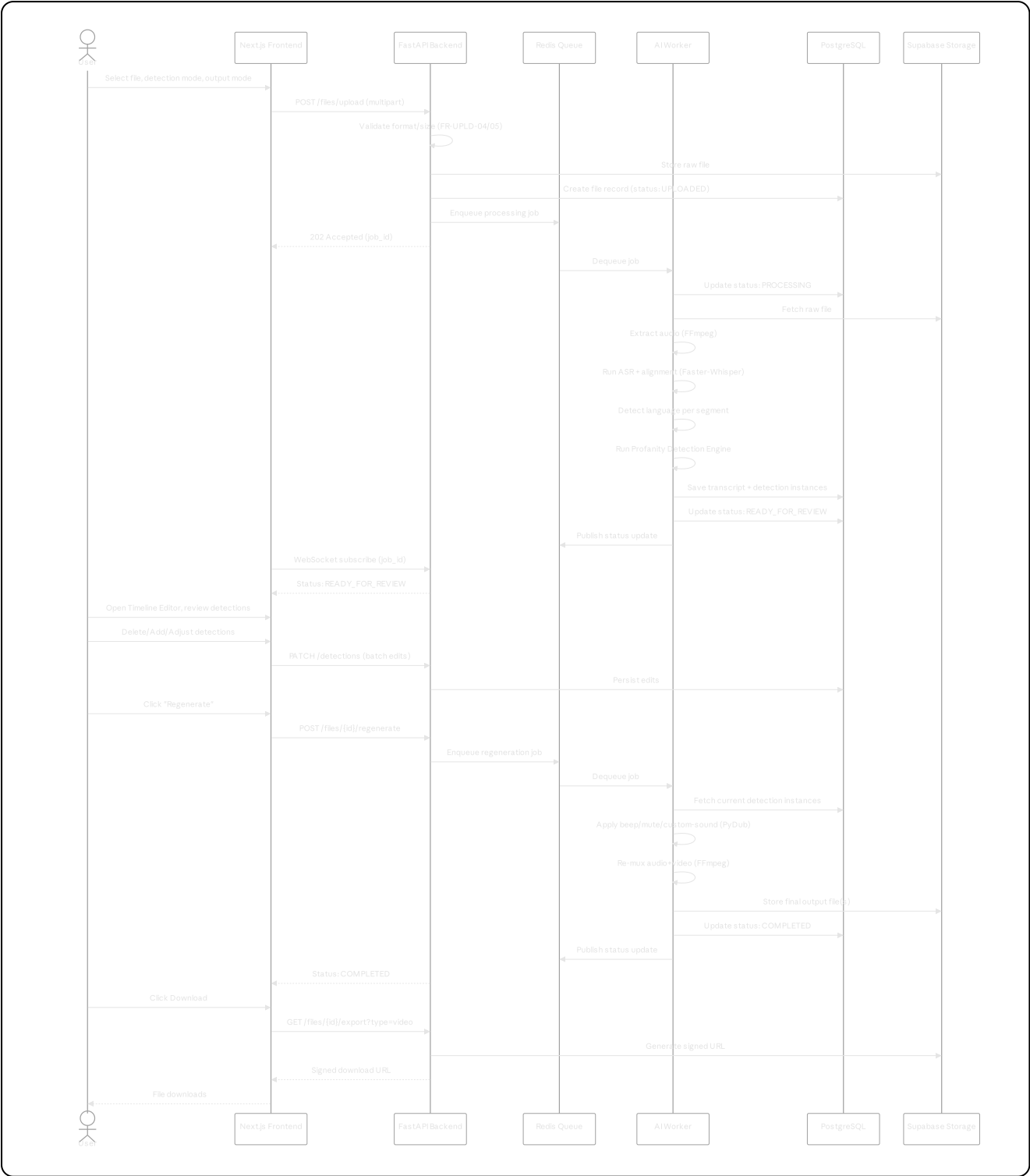


3.1 Component Responsibilities

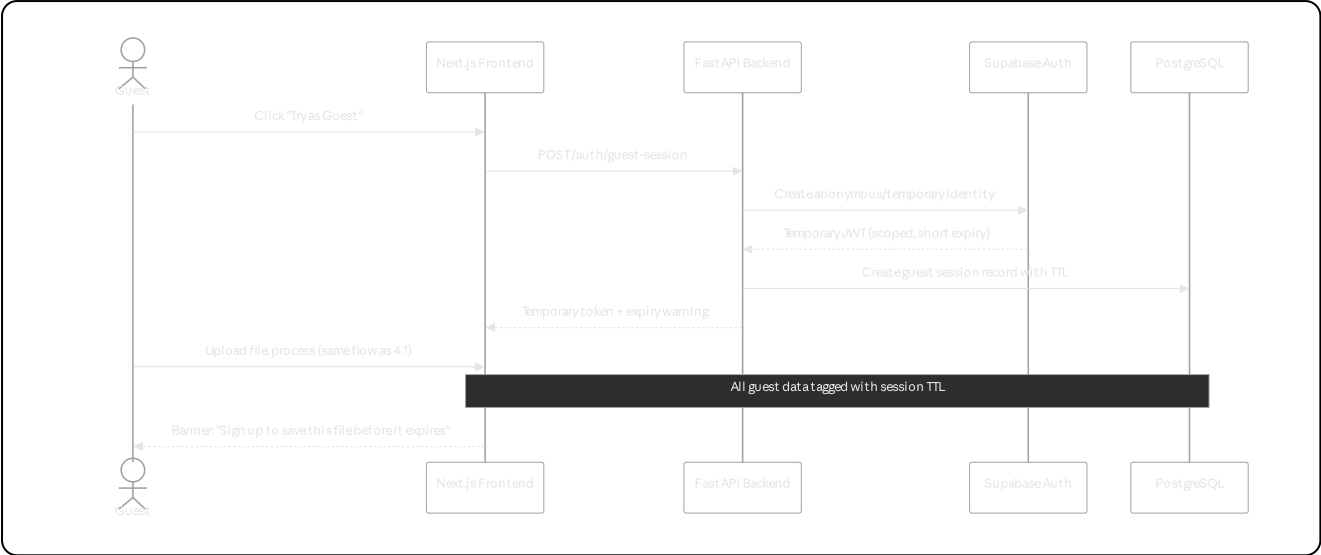
Component	Responsibility
AuthService	Signup/login, token issuance, guest session management
FileService	Upload validation, file metadata management, storage interaction
JobOrchestratorService	Creates processing jobs, enqueues to Redis, tracks job state
DetectionConfigService	Manages detection mode selection, custom word lists, presets
TimelineEditService	Handles CRUD on detection instances during user review/edit
ExportService	Assembles final downloadable artifacts (video/audio/srt/txt/json)
AdminService	User management, profanity dictionary management, analytics, logs
SpeechRecognitionService	Abstracts the ASR engine (Faster-Whisper); swappable interface
LanguageDetectionService	Determines language(s) per segment
ProfanityDetectionService	Runs the hybrid detection pipeline (normalize → dict/regex → embedding scoring)
AudioEditingService	Applies beep/mute/custom-sound edits to specific timestamp ranges
MediaMuxService	Re-combines edited audio with original video via FFmpeg

4. Sequence Diagrams

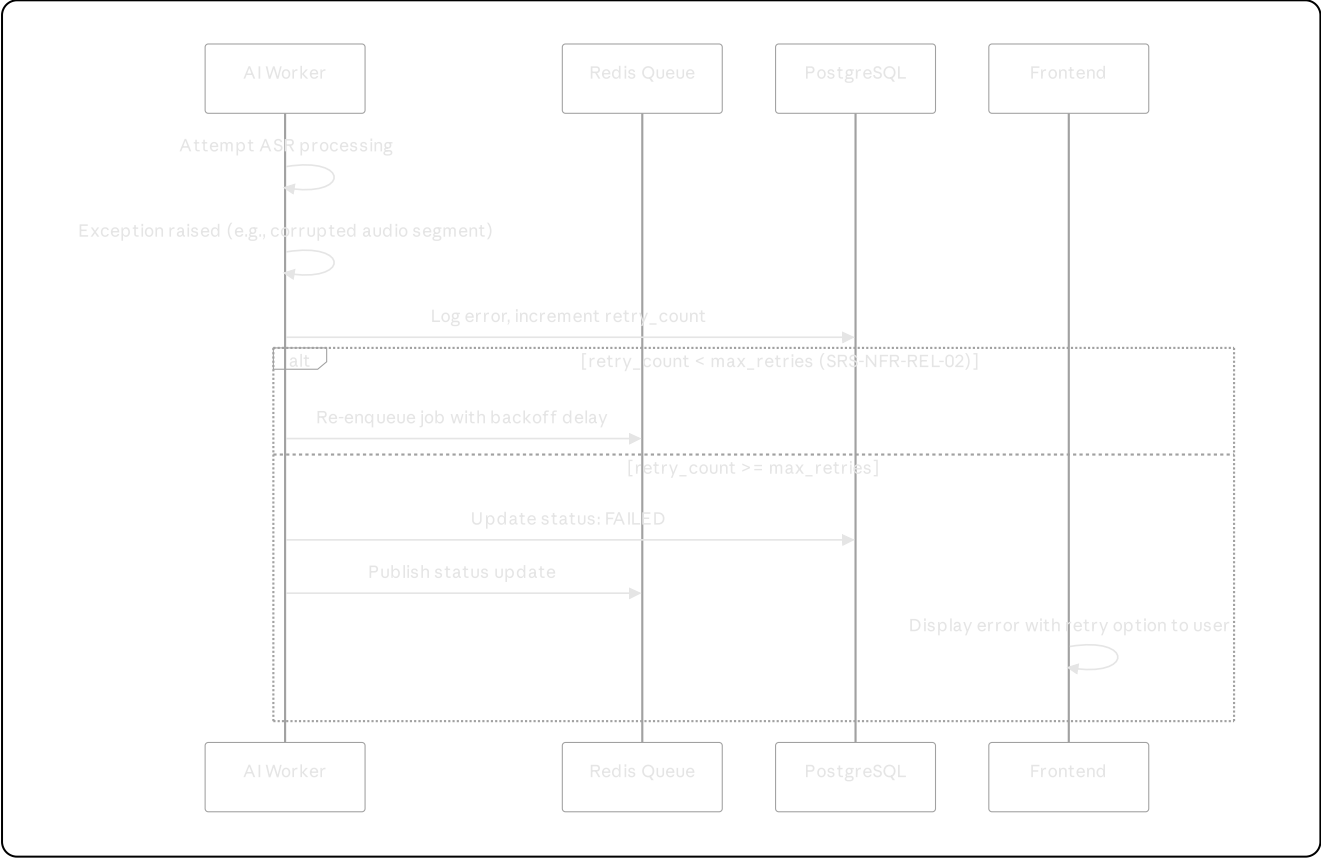
4.1 Primary Flow: Upload → Processing → Review → Export



4.2 Secondary Flow: Guest Mode Upload

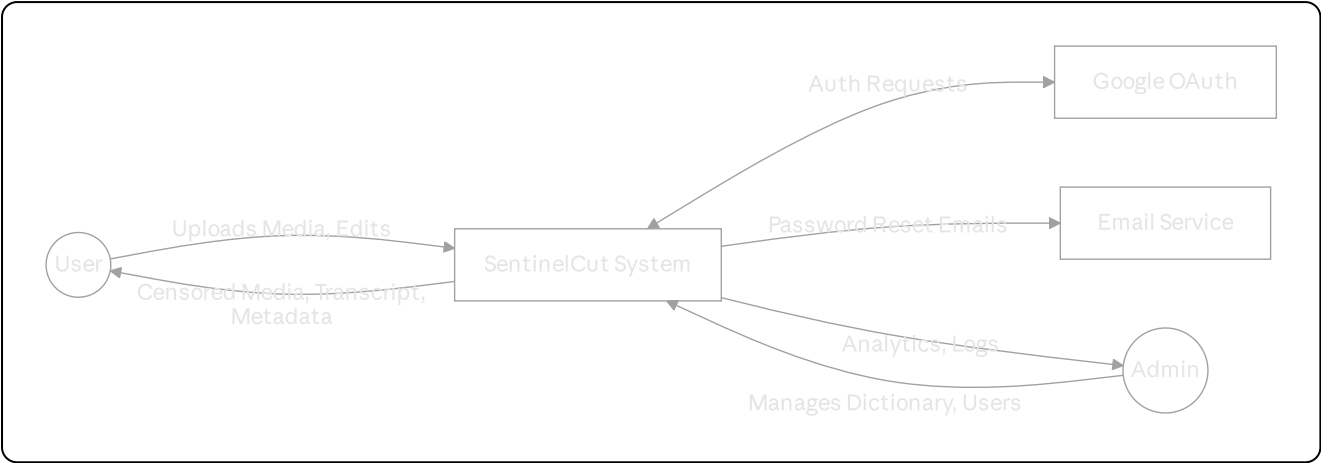


4.3 Edge Case Flow: Failed ASR / Retry

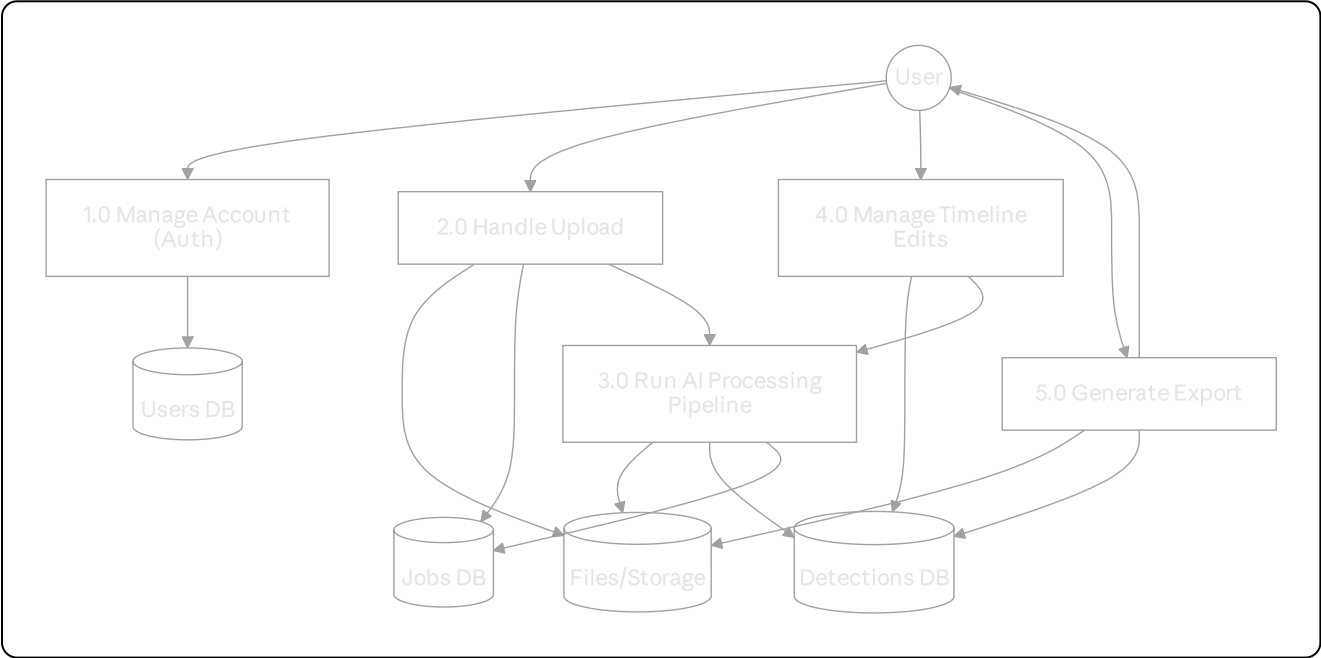


5. Data Flow Diagrams (DFD)

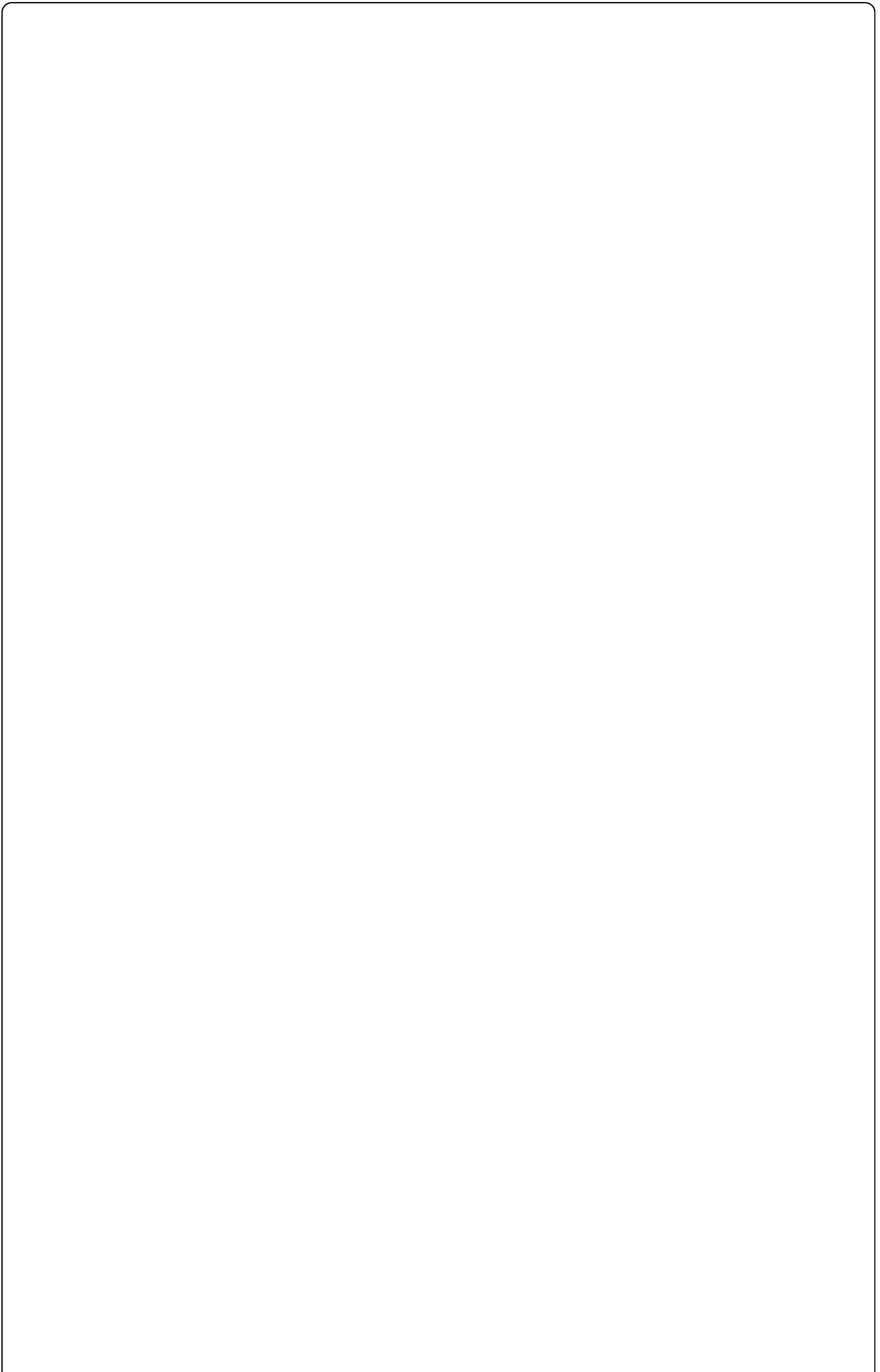
5.1 DFD Level 0 (Context Diagram)

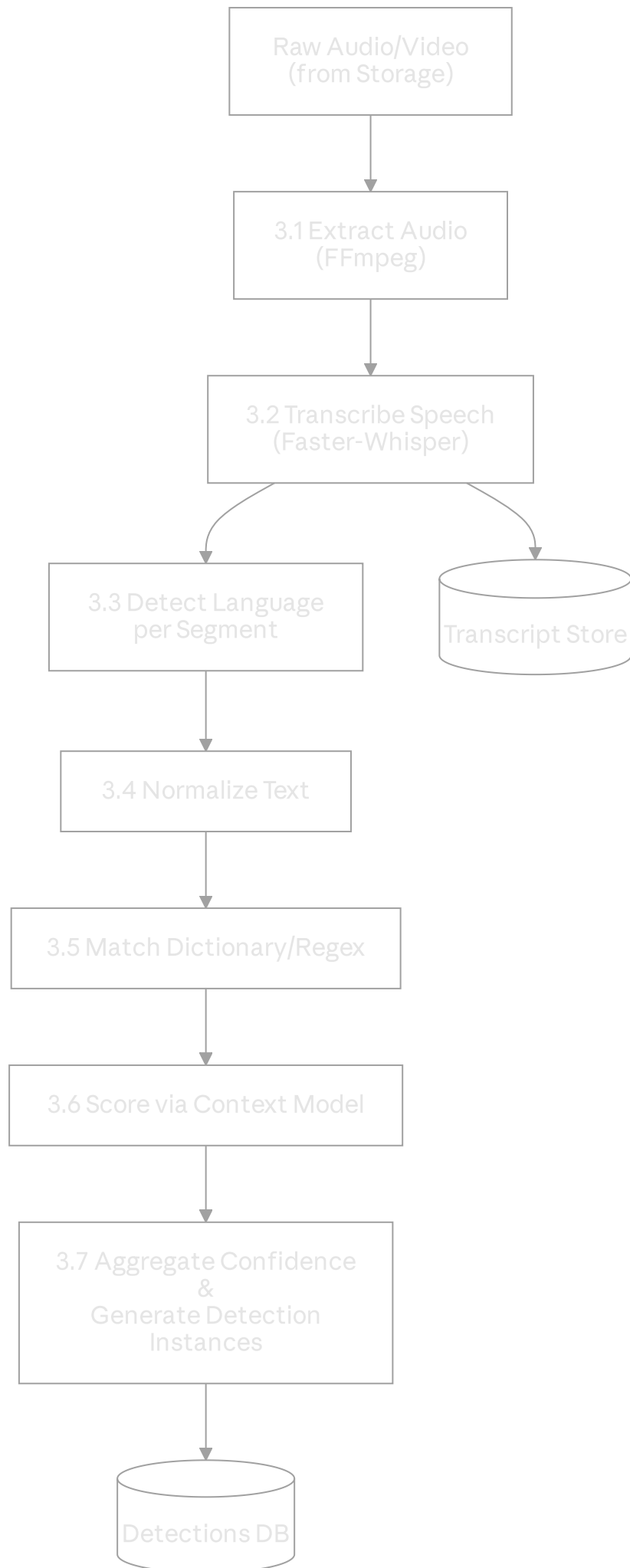


5.2 DFD Level 1 (Major Processes)



5.3 DFD Level 2 (Decomposition of Process 3.0 — AI Processing Pipeline)





6. Deployment Diagram

flowchart TB

```
    subgraph UserDevice["User Device"]
```

```
        BROWSER["Web Browser"]
```

```
    end
```

```
    subgraph VercelCloud["Vercel (Frontend Hosting)"]
```

```
        NEXTAPP["Next.js App<br/>(SSR + Static Assets)"]
```

```
    end
```

```
    subgraph RenderCloud["Render / Railway (Backend Hosting)"]
```

```
        APIINST["FastAPI Instance(s)"]
```

```
        WORKERINST["Celery/RQ Worker Instance(s)"]
```

```
    end
```

```
    subgraph UpstashCloud["Upstash (Managed Redis)"]
```

```
        REDISINST["Redis Instance"]
```

```
    end
```

```
    subgraph SupabaseCloud["Supabase (Managed Platform)"]
```

```
        AUTHINST["Supabase Auth"]
```

```
        PGINST["PostgreSQL Instance"]
```

```
        STORAGEINST["Object Storage Buckets"]
```

```
    end
```

```
    subgraph ThirdParty["Third-Party Services"]
```

```
        GOOGLEOAUTH["Google Identity Platform"]
```

```
        EMAILSVC["Email API (e.g., Resend)"]
```

```
    end
```

```
BROWSER <-->|HTTPS| NEXTAPP
```

```
BROWSER <-->|HTTPS / WSS| APIINST
```

```
NEXTAPP -->|API Calls (HTTPS)| APIINST
```

```
APIINST <-->|SQL| PGINST
```

```
APIINST <-->|Auth Calls| AUTHINST
```

```
APIINST -->|Enqueue| REDISINST
```

```
WORKERINST -->|Dequeue| REDISINST
```

```
WORKERINST <-->|Read/Write Files| STORAGEINST
```

```
WORKERINST -->|Write Results| PGINST
```

```
AUTHINST <--> GOOGLEOAUTH
```

```
APIINST --> EMAILSVC
```

6.1 Deployment Notes

Concern	Approach
Scaling AI Workers	Worker instances can be scaled horizontally (multiple replicas consuming the same Redis queue) as load increases — this is the primary bottleneck to monitor
Zero-downtime deploys	Frontend (Vercel) and backend (Render/Railway) both support rolling deploys independently
Environment separation	Separate <code>staging</code> and <code>production</code> environment configs (Supabase projects, Redis instances) recommended even for academic demo, to avoid corrupting demo data during development
Secrets management	API keys/DB credentials stored as environment variables in each platform's secret manager — never committed to source control

7. Cross-Cutting Concerns

7.1 Security in the Architecture

- JWT validated at the `AuthMiddleware` layer before any request reaches business logic (satisfies SRS-NFR-SEC-02).
- Signed, time-limited URLs used for all direct file upload/download to/from Supabase Storage — the backend never proxies large binary payloads through itself unnecessarily.
- RBAC enforced at the service layer: `AdminService` methods check role claims from the JWT before executing.

7.2 Observability

- Each job (upload, processing, regeneration) is logged with a `job_id`, timestamps, and status transitions in the `Jobs` table, feeding both the user-facing processing queue UI (SRS-FR-DASH-05) and the Admin analytics view (SRS-FR-ADM-04).
- Structured logging (JSON logs) at the API and Worker layers to support future integration with a log aggregation tool.

7.3 Extensibility Points (Explicitly Designed For)

- `SpeechRecognitionService` interface → swap Faster-Whisper for a cloud ASR API in a future paid tier without touching callers.
- `ProfanityDetectionService` interface → the embedding/context-scoring model can be upgraded or replaced independently of the dictionary/regex layer.
- `AudioEditingService` → designed to accept a future `VoiceReplacementStrategy` (AI voice cloning) as an additional replacement mode without restructuring the

pipeline.

8. Next Document

Document 6: Database Design — full ER diagram, table schemas, relationships, indexing strategy, and constraints for the PostgreSQL/Supabase data layer defined architecturally above.

End of System Architecture Document